

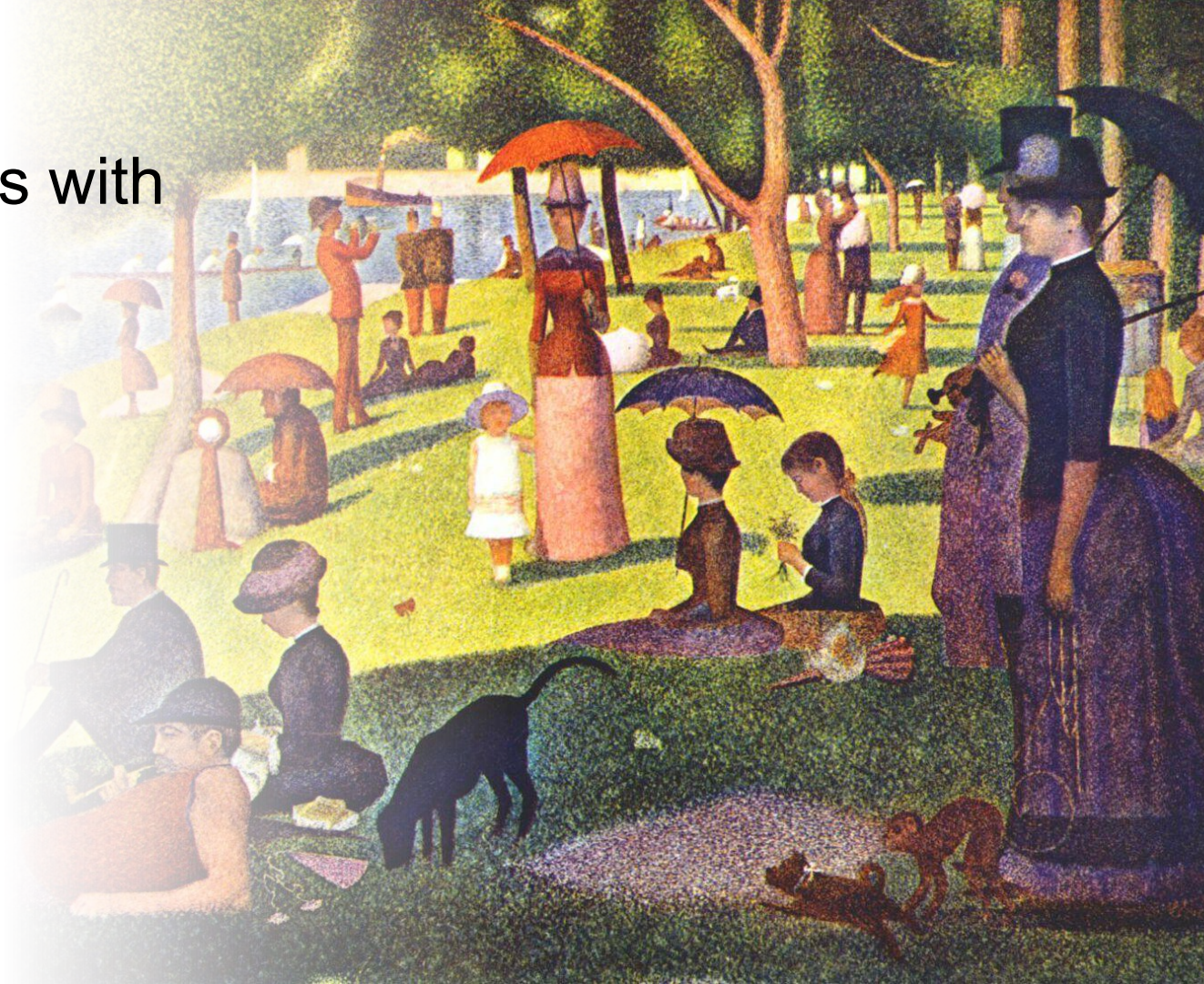
Single Cell RNA Seq Analysis with Seurat and R

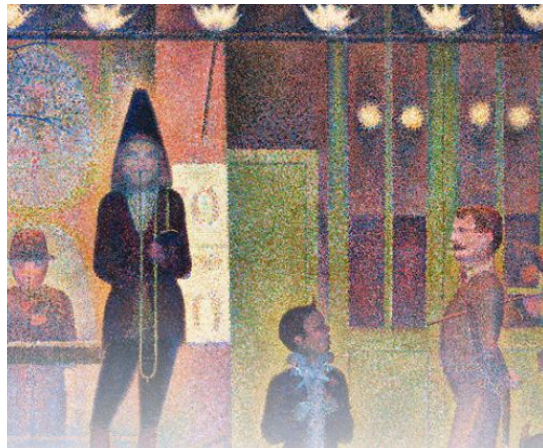
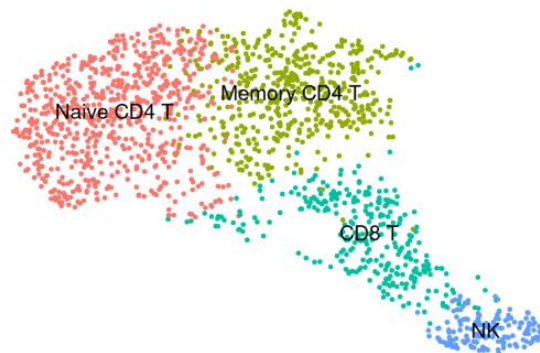
March 26, 2026

Steve Rozen

steverozen [at] pm.me

sr110 [at] duke.edu





Workshop objectives:

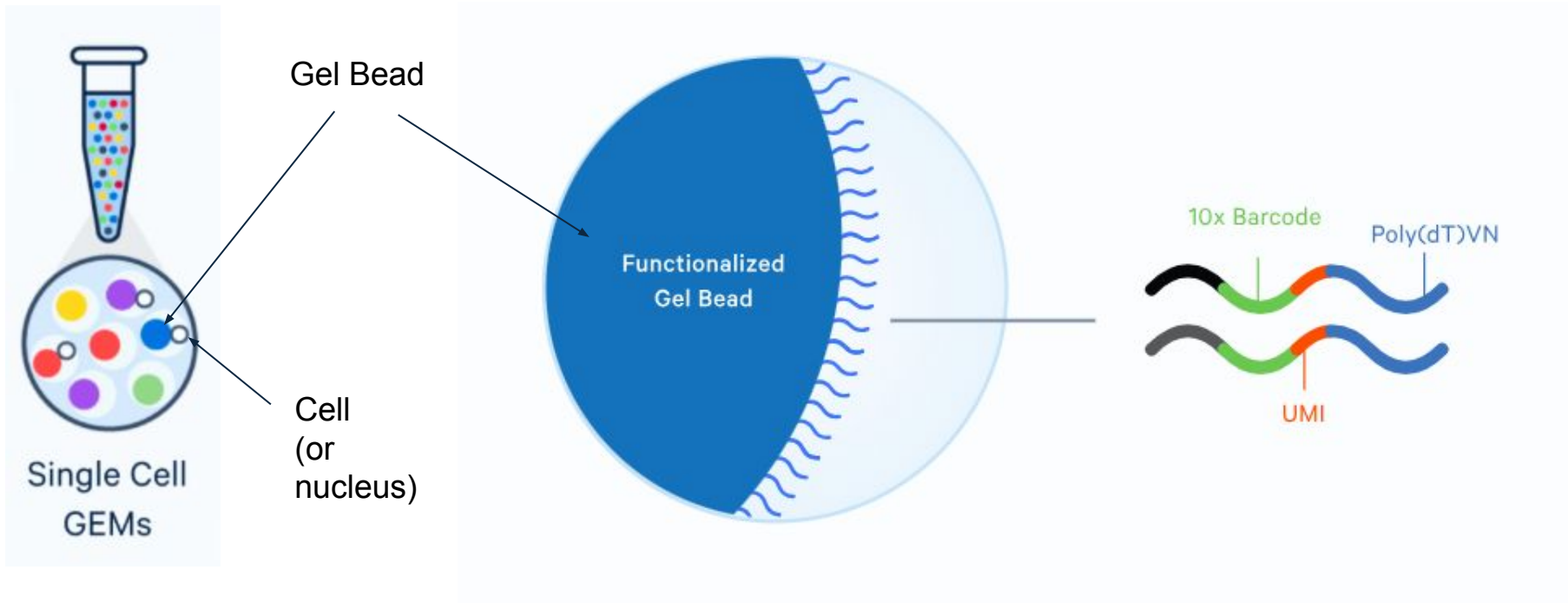
Understand a simple single cell RNA-seq experiment and analyze it with a commonly-used workflow

Gain experience working in R and and IDE such as RStudio, Positron, VSCode

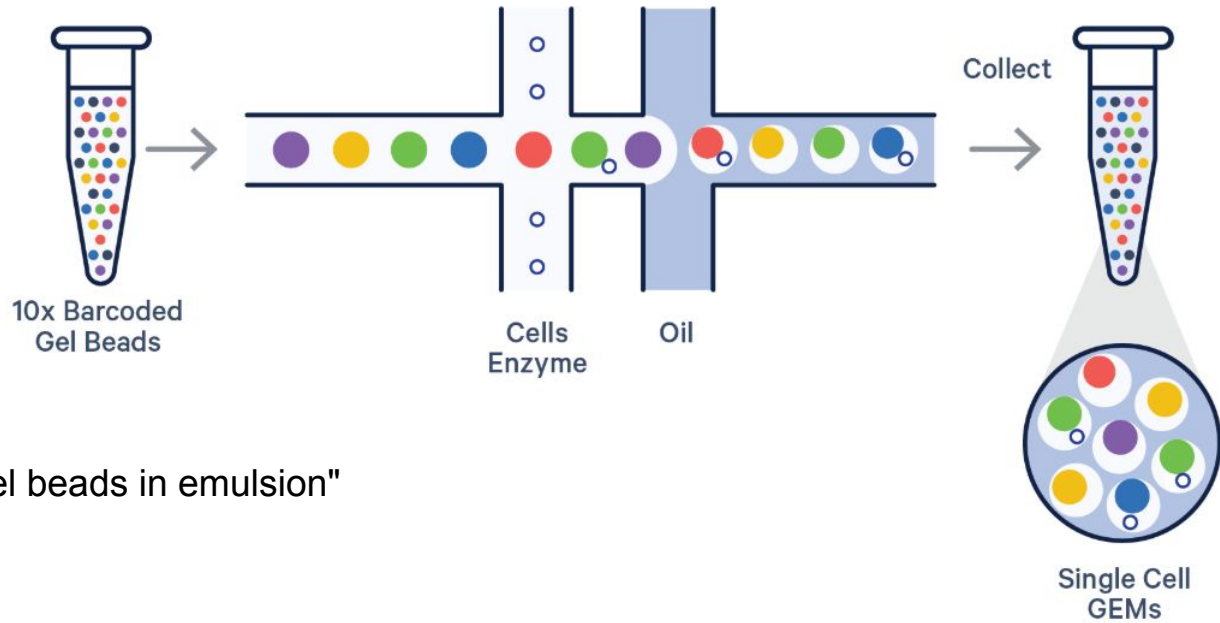
importance of conceptual orientation rather than details
know what is possible, ask LLM how to do it

Gain experience in the Seurat ecosystem for RNA-seq analysis

Where the data come from...



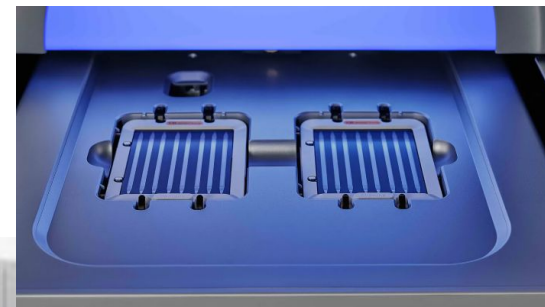
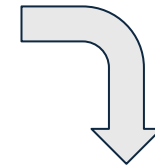
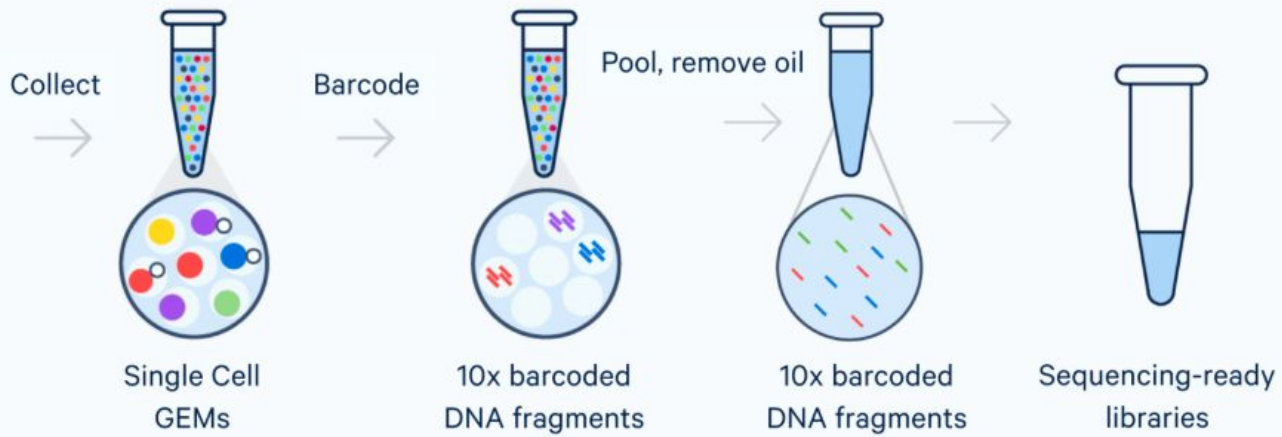
<https://www.10xgenomics.com/platforms/chromium/technology>



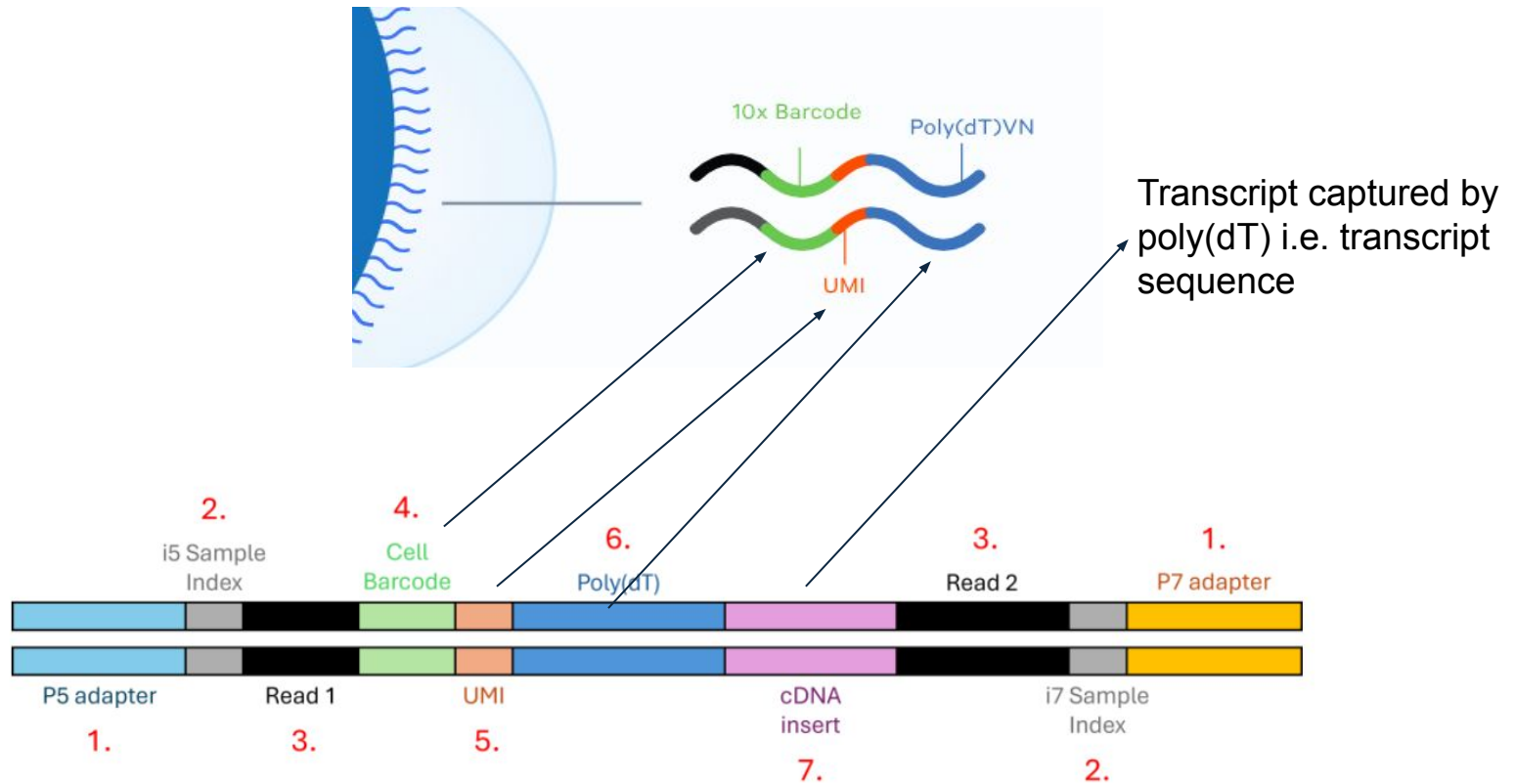
GEM = "Gel beads in emulsion"

https://cdn.10xgenomics.com/image/upload/f_auto,q_auto,w_900,dpr_2.0/v1709753672/blog/GEM-X%20Launch%20blog/vertical_flipped_GEM96_H1975_high_4_S0001.gif

<https://www.10xgenomics.com/blog/the-next-generation-of-single-cell-rna-seq-an-introduction-to-gem-x-technology>



Short-read sequencer



Each UMI comes from 1 input cDNA molecule
 Each Cell Barcode is 1 cell/nucleus from a droplet

The sequencer generates "BCL" files

Downstream processing needs fastq files

For single cell RNA seq we often used 10X's cellranger software

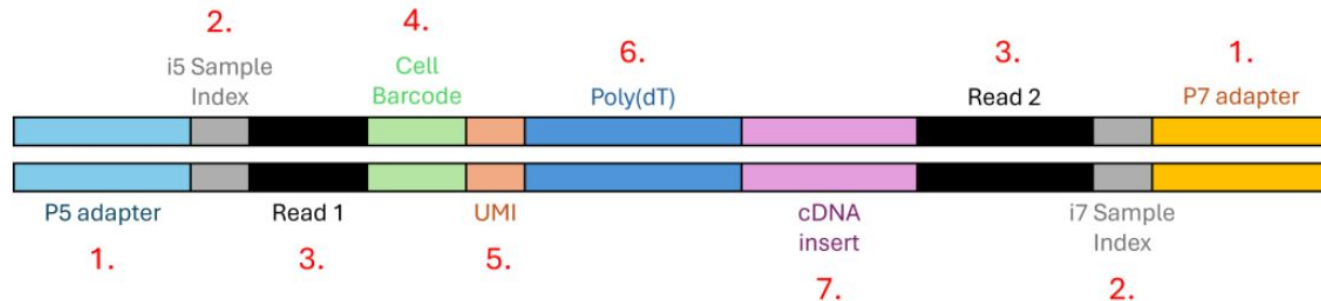
- **cellranger mkfastq** demultiplexes raw base call (BCL) files generated by Illumina sequencers into FASTQ files. It is a wrapper around Illumina's bcl2fastq, with additional features that are specific to 10x Genomics libraries and a simplified sample sheet format.

FASTQ consists of many groups of 4 lines like this:

```
@SEQ ID
GATTTGGGGTTCAAAGCAGTATCGATCAAATAGTAAATCCATTTGTTCAACTCACAGTTT
+
! '*' ( ( ( (***) ) %%%++) (%%%) .1***-+*' ' ) **55CCF>>>>>CCCCCCC65
```

<https://support.10xgenomics.com/single-cell-gene-expression/software/pipelines/latest/what-is-cell-ranger>

[illegible]



- **cellranger count** takes FASTQ files from `cellranger mkfastq` and performs alignment, filtering, barcode counting, and UMI counting. It uses the Chromium cellular barcodes to generate feature-barcode matrices, determine clusters, and perform gene expression analysis. The `count` pipeline can take input from **multiple sequencing runs** on the same **GEM well**. `cellranger count` also processes **Feature Barcode** data alongside Gene Expression reads.

<https://biotech.wisc.edu/guide-to-getting-started-with-10x-genomics-single-cell-rna-seq>

<https://support.10xgenomics.com/single-cell-gene-expression/software/pipelines/latest/what-is-cell-ranger>

For our tutorial, get output from cellranger count at

https://github.com/steverozen/single-cell-2026/raw/refs/heads/main/pbmc3k_filtered_gene_bc_matrices.tar.gz

From R:

```
download.file("https://github.com/steverozen/single-cell-2026/raw/refs/heads/main/pbmc3k_filtered_gene_bc_matrices.tar.gz", destfile = "x.tar.gz")  
  
untar("x.tar.gz")
```

Input: Public 10X genomics data set

(Now you need to register to access this; not sure if the link still works after that)

<https://www.10xgenomics.com/datasets/3-k-pbm-cs-from-a-healthy-donor-1-standard-1-1-0>

Processed with 10X Cell Ranger software

<https://www.10xgenomics.com/support/software/cell-ranger/latest>

 **Dataset overview**

 **Output and supplemental files**

 **Input files**

Peripheral blood mononuclear cells (PBMCs) from a healthy donor (same donor as pbmc6k).

PBMCs are primary cells with relatively small amounts of RNA (~1pg RNA/cell).

- 2,700 cells detected
- Sequenced on Illumina NextSeq 500 with ~69,000 reads per cell
- 98bp read1 (transcript), 8bp I5 sample barcode, 14bp I7 GemCode barcode and 10bp read2 (UMI)
- Analysis run with --cells=3000

Product

Universal 3' Gene Expression

Software

Cell Ranger

Subpipeline

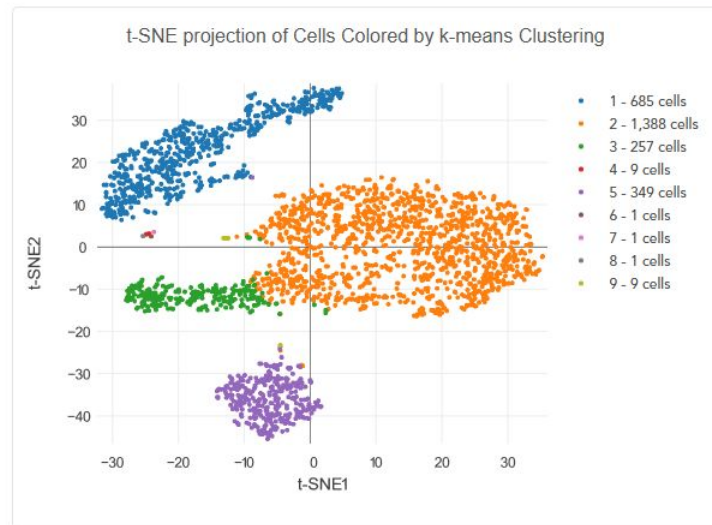
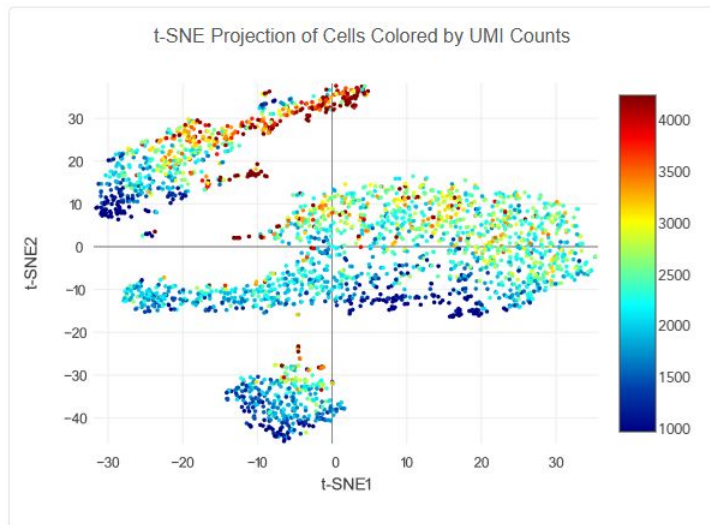
cellranger run

10x Instrument(s)

Chromium Controller

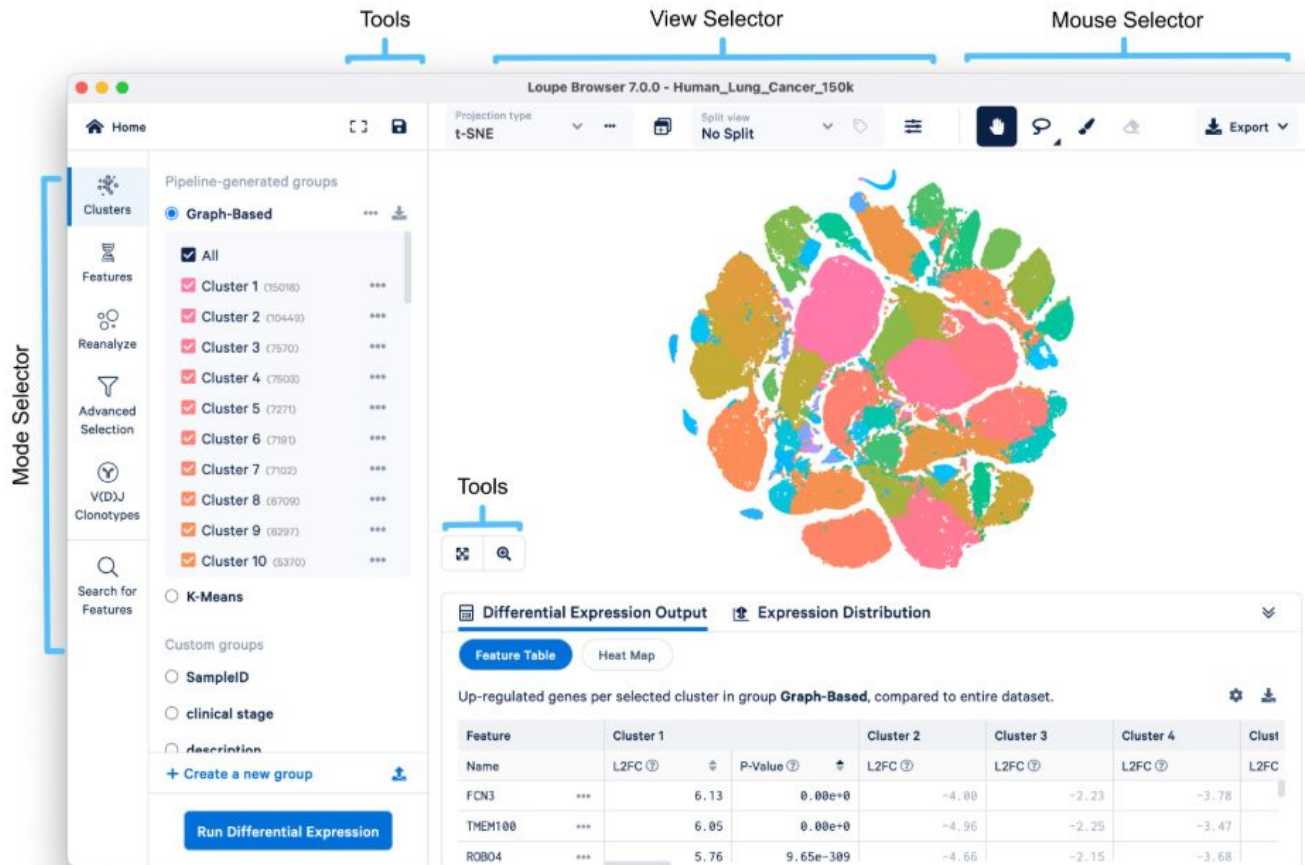
Cell Ranger output on this data set:

https://cf.10xgenomics.com/samples/cell-exp/1.1.0/pbmc3k/pbmc3k_web_summary.html



Top Differentially Expressed Genes per Cell Cluster (UMI counts/cell)										
Gene IDs	Gene names	1	2	3	4	5	6	7	8	9
ENSG00000101439	CST3	16.58	0.22	0.26	1.67	0.25	2.00	14.00	1.00	0.89
ENSG00000011600	TYROBP	12.47	0.16	2.09	0.00	0.13	0.00	26.00	0.00	1.22
ENSG00000090382	LYZ	38.39	0.69	0.59	0.67	0.64	0.00	10.00	0.00	2.11
ENSG00000087086	FTL	90.20	6.16	5.40	5.33	7.13	4.00	220.00	11.00	22.78
ENSG00000204482	LST1	8.02	0.19	0.12	0.00	0.14	0.00	19.00	0.00	0.78
ENSG00000204472	AIF1	8.16	0.30	0.09	0.11	0.09	0.00	17.00	0.00	1.44

There is also 10x Loupe Browser desktop app



If Cell Ranger and Loupe Browser can do a lot,
why are you interested in learning how to do the analysis in R?

Possible answers a-priori:

Get deeper conceptual understanding; really understand the sausage making

Learn generic skills applicable for other data analysis tasks

Added flexibility for follow-on analysis (trajectory [Monocle], RNA velocity, different approaches to data set integration and batch correction)

Add to CV

R / Seurat is the standard

Not added by workshop attendees:

Customized graphics

SINGLE CELL RNA SEQ WORKFLOW

<https://github.com/satijalab/seurat/wiki/Seurat>

Read count data and CreateSeuratObject

QC and select cells for further analysis

Normalize the data

Select highly variable features (genes)

Scale the data (mean = 0, variance = 1)



Linear dimensionality reduction = PCA

Determine 'dimensionality' of the data set



Cluster the cells

Non-linear dimensionality reduction (UMAP)

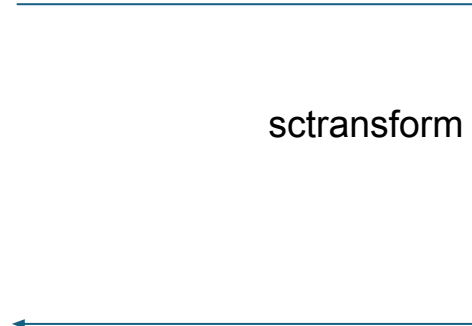
Differentially abundant features (i.e. genes) == cluster biomarkers

Determine cell-type identity

Alternative pathway
(recommended)

sctransform normalization

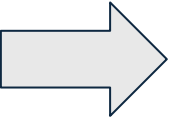
https://satijalab.org/seurat/articles/sctransform_vignette



More ways to determine cell identity, some with public marker sets

<https://share.google/aimode/EkWxoC4SQ5vbJjhvo>

We will review two tutorials from the Satija lab
– creators of Seurat

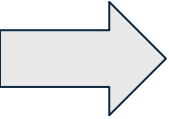


https://satijalab.org/seurat/articles/pbmc3k_tutorial

https://satijalab.org/seurat/articles/sctransform_vignette

We will review two tutorials from the Satija lab
– creators of Seurat

https://satijalab.org/seurat/articles/pbmc3k_tutorial



https://satijalab.org/seurat/articles/sctransform_vignette

Participant query: the tutorial gave us marker genes and cell type assignments (https://satijalab.org/seurat/articles/pbm3k_tutorial#assigning-cell-type-identity-to-clusters). How would we do these on our own?

We demo'ed Claude Code doing this.

Conversation history:

<https://raw.githubusercontent.com/steverozen/single-cell-2026/refs/heads/main/get-cell-types-claude-code-demo.html>

History formatted by clogviewer:

<https://github.com/Brads3290/clogviewer>

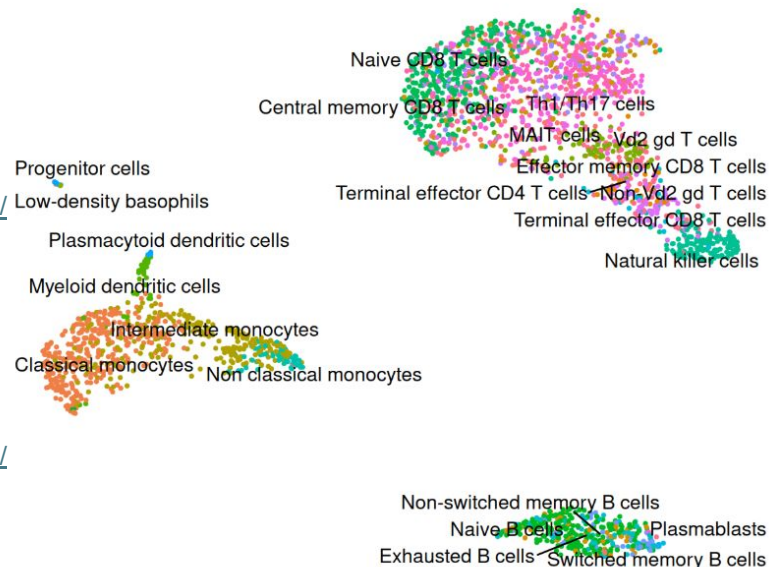
Final output:

<https://raw.githubusercontent.com/steverozen/single-cell-2026/refs/heads/main/cell-type-annotation.html>

Source Quarto file generated by Claude Code:

<https://github.com/steverozen/single-cell-2026/blob/main/cell-type-annotation.qmd>

SingleR (Monaco, fine)





PCA deep dive

Principal Component Analysis

For us, each point is a cell

Dimension 1 (x axis) is level of gene A

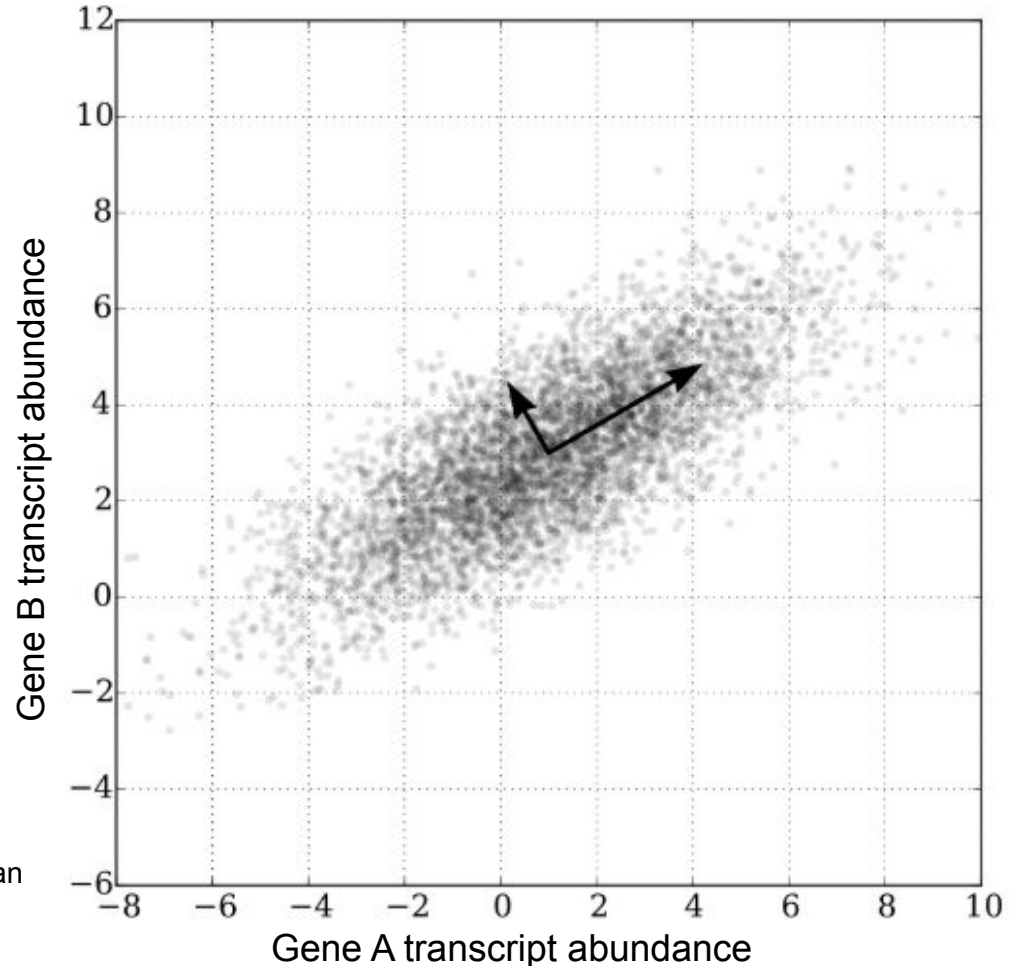
Dimension 2 (y axis) is level of gene B

Dimension 3 (not show)

....

Dimension 1000

....



https://en.wikipedia.org/wiki/Principal_component_analysis#/media/File:GaussianScatterPCA.svg



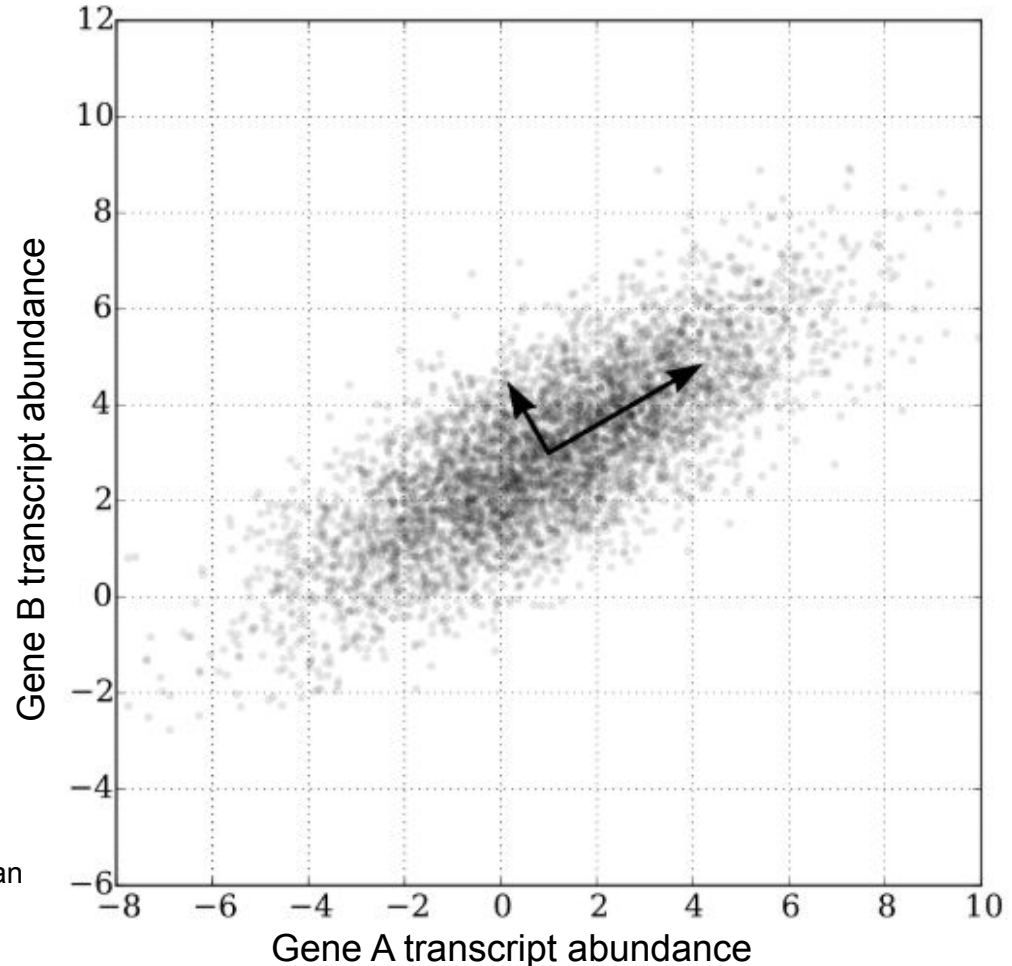
PCA deep dive

1st principal component is longest arrow and explains the most variance in gene transcript abundance

2nd principal component is the next longest arrow, *at right angles* to 1st components
explains the most remaining variance

You can use this to put each cell into a lower dimensional coordinate system

(Up to you to decide how long to keep adding principal components)



https://en.wikipedia.org/wiki/Principal_component_analysis#/media/File:GaussianScatterPCA.svg



PCA deep dive

Principal Component Analysis

For us, each point is a cell

Dimension 1 (x axis) is level of gene A

Dimension 2 (y axis) is level of gene B

Dimension 3 (not show)

....

Dimension 1000

....

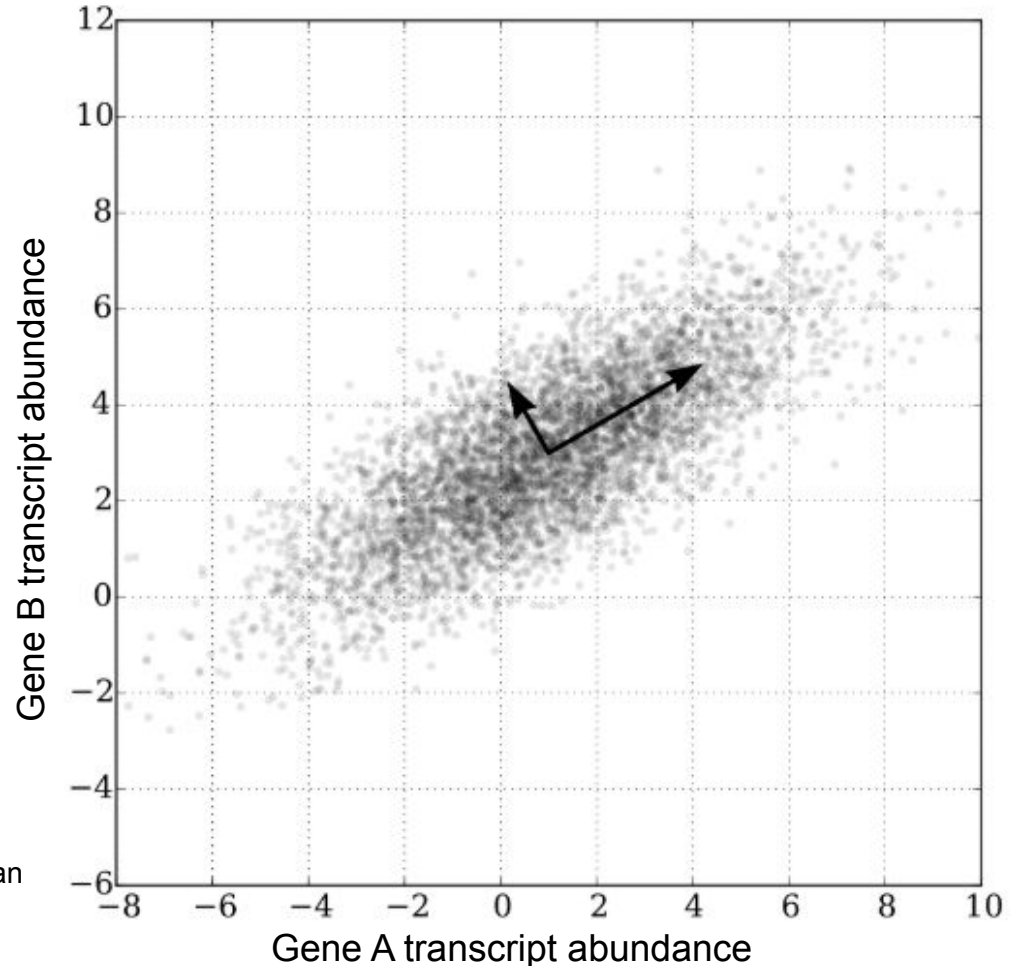
Gene A (x axis) contributes more to PC1
than Gene B)

Gene B (y axis) contributes more to PC2

...

"Loading"

https://en.wikipedia.org/wiki/Principal_component_analysis#/media/File:GaussianScatterPCA.svg



What is Louvain Clustering?



Definition

Louvain is a greedy, hierarchical algorithm for detecting communities (clusters) in networks.

Developed in 2008 by Blondel et al. at the University of Louvain (Belgium).

It maximizes a quality function called **Modularity** — measuring how densely nodes are connected within communities vs. what would be expected randomly.



Speed

Handles millions of nodes in seconds



Scalable

Sub-quadratic time complexity



Hierarchical

Produces a dendrogram of communities



No k needed

Number of clusters is auto-discovered

Modularity of Weighted Graph: The Quality Function

$$Q = (1/2m) \sum_i \sum_j [w_{ij} - s_i s_j / 2m] \cdot \delta(c_i, c_j)$$

w_{ij} Edge weight between nodes i and j (0 if no edge)

$\delta(c_i, c_j)$ 1 if nodes i and j are in the same community, else 0

s_i, s_j Strength (weighted degree) of nodes i and j : $s_i = \sum_j w_{ij}$

Q Modularity score (ranges from -0.5 to 1; higher = better communities)

m Total edge weight in graph: $m = \frac{1}{2} \sum_i s_i$

Reduces to the unweighted form when all $w_{ij} \in \{0,1\}$: then $s_i = k_i$ and $m = |E|$.

Modularity of Unweighted Graph: Quality Function



$$Q = (1/2m) \sum_{i,j} [A_{ij} - k_i k_j / 2m] \cdot \delta(c_i, c_j)$$

A_{ij}

Adjacency matrix: 1 if edge exists, 0 otherwise

$\delta(c_i, c_j)$

1 if nodes i and j are in the same community, else 0

k_i, k_j

Degree of nodes i and j

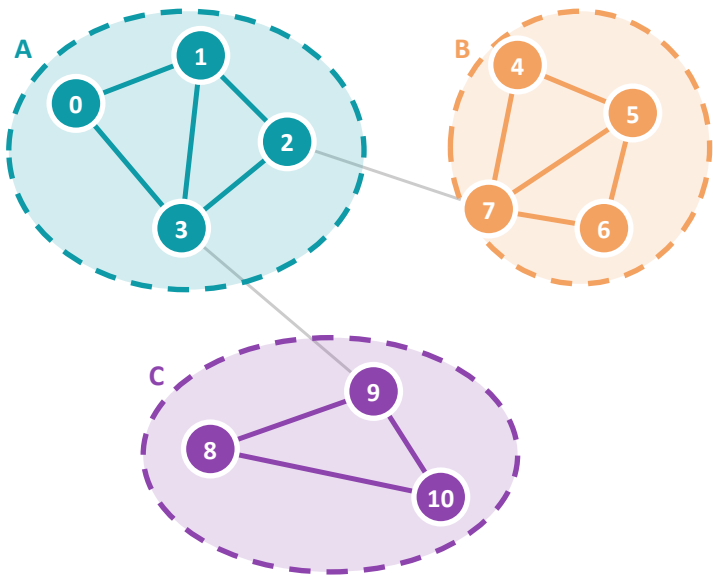
Q

Modularity score (ranges from -0.5 to 1; higher = better communities)

m

Total number of edges in the graph

Modularity in Practice: High Q



$$Q \approx 0.67$$

HIGH modularity

✓ Dense intra-community edges

Each community has many internal connections (5 edges within A, 5 within B, 3 within C).

✓ Sparse inter-community edges

Only 2 edges cross between communities — far fewer than expected by chance.

✓ $A_{ii} - k_i k_i / 2m > 0$

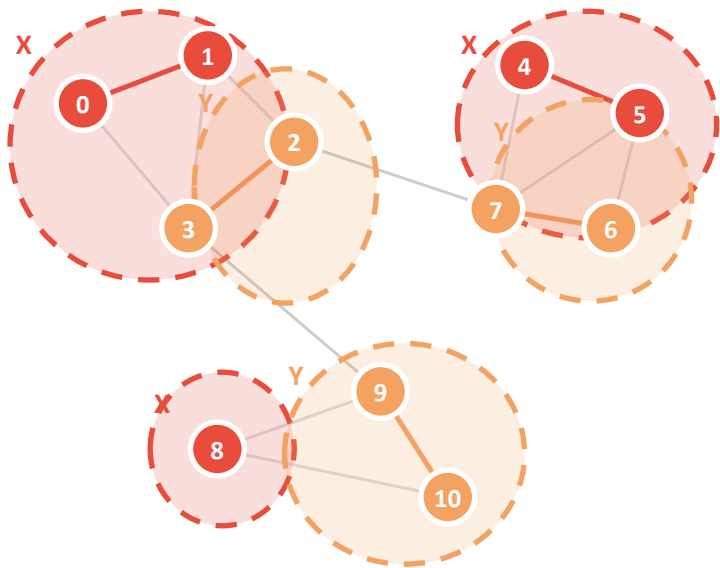
For same-community pairs, observed edges exceed the random expectation. Each term contributes positively to Q.

✓ Clear cluster separation

Community boundaries are obvious: removing the 2 bridge edges would fully disconnect the graph.

Intuition: most edges are inside communities; very few cross between them. Q rewards this structure.

Modularity in Practice: Low Q



$$Q \approx -0.09$$

LOW modularity

✗ Communities split real clusters

Each of the 3 natural clusters (A, B, C) is sliced in half — nodes 0,1 are in X while 2,3 are in Y.

✗ Many edges cross boundaries

Most of the original intra-cluster edges now run between X and Y, not within them.

✗ $A_i - k_i^2/2m < 0$ often

Observed edges are fewer than the random expectation for same-community pairs. Each term subtracts from Q.

✗ No benefit over random

A Q near 0 (or negative) means the partition is no better — or worse — than assigning communities at random.

Intuition: edges mostly cross community boundaries. The partition "wastes" connections — Q penalises this.

The Louvain Algorithm: Two Phases



PHASE 1 — Modularity Optimization

1. Assign each node to its own community.
2. For each node i , try moving it to each neighboring community in turn (neighbors visited in random order).
3. Compute the ΔQ gain for each candidate move.
4. Apply the move with the highest $\Delta Q > 0$; skip if none improve.
5. Repeat until no single move improves Q .

PHASE 2 — Network Aggregation

1. Compress all nodes in a community into a single 'super-node'.
2. Edge weights between super-nodes = sum of original edges.
3. Self-loops represent internal edges of the community.
4. Apply Phase 1 again on this compressed graph.
5. Repeat both phases until no further improvement.

Computing the Modularity Gain (ΔQ)



When moving node i into community C :

$$\Delta Q = [\sum_{i \in C} w_{ij} + 2k_{i,C}] / 2m - [(\sum_{i \in C} d_i + k_i) / 2m]^2 - \sum_{i \in C} w_{ij} / 2m + (\sum_{i \in C} d_i / 2m)^2 + (k_i / 2m)^2$$

$\sum_{i \in C} w_{ij}$

Sum of weights of edges inside community C

$k_{i,C}$

Sum of weights of edges from i to nodes in C

$\sum_{i \in C} d_i$

Sum of degrees of all nodes in C

m

Total weight of all edges in graph

k_i

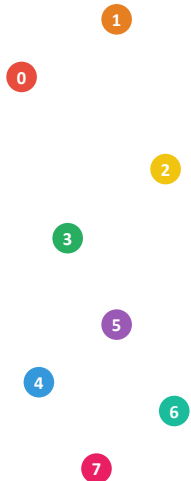
Degree (sum of edge weights) of node i

Only move node i if $\Delta Q > 0$

Louvain in Action: Visual Walkthrough

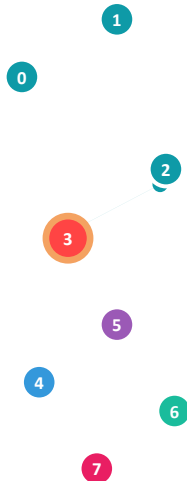


Step 1



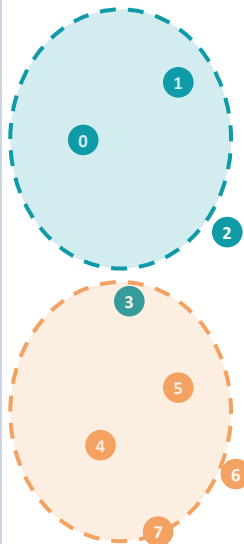
Each node in
its own community

Step 2



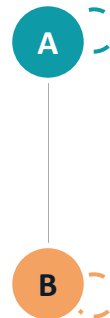
Node moves to
best neighbor community

Step 3



Phase 1 complete:
communities formed

Phase 2



Communities collapsed
to super-nodes

Iterate



No ΔQ gain $\rightarrow$
converged!

Phase 1 re-runs
on super-node graph

◀ Phase 1: Modularity Optimization ▶

◀ Phase 2 ▶

◀ Repeat ▶

Complexity & Key Properties



$O(n \log n)$

Time Complexity

Nearly linear in practice

$O(n)$

Space Complexity

Only node/edge lists needed

0 to 1

Modularity Range

Higher values = clearer communities

✓ Strengths

- Fast on massive graphs (millions of nodes)
- No need to pre-specify k (number of clusters)
- Produces hierarchical community structure
- Widely implemented (NetworkX, igraph, etc.)

✗ Limitations

- Resolution limit: misses small communities in large graphs
- Non-deterministic (random initialization)
- Assumes undirected, weighted graphs by default
- Can vary across runs

Real-World Applications



Social Networks

Detect friend groups, echo chambers, and influencer communities on Twitter/Facebook

Bioinformatics

Identify protein interaction modules and gene co-expression clusters

Finance

Detect correlated stock clusters and fraud rings in transaction networks

NLP / Knowledge

Build topic clusters and entity graphs from document co-occurrence networks

Infrastructure

Partition power grids and telecom networks for resilience analysis

Recommendation

Group users with similar tastes in collaborative filtering systems

Python Implementation



Python • python-louvain (community)

```
import networkx as nx
import community as community_louvain
import matplotlib.pyplot as plt

# Create or load a graph
G = nx.karate_club_graph()

# Run Louvain clustering
partition = community_louvain.best_partition(G)

# partition = {node_id: community_id, ...}
print(f'Communities found: {max(partition.values())+1}')

# Visualize
pos = nx.spring_layout(G)
nx.draw(G, pos, node_color=list(partition.values()),
        cmap=plt.cm.tab20, with_labels=True)
plt.show()
```

Install

```
pip install python-louvain networkx
matplotlib
```

resolution param

Controls cluster granularity (default=1). Higher = smaller clusters.

randomize

Pass randomize=False for reproducible results (slower).

weighted graphs

Edge 'weight' attribute is used automatically if present.

Key Takeaways Louvain Clustering



- 01 Louvain maximizes modularity Q by greedily reassigning nodes and aggregating communities.
- 02 Two phases repeat: local optimization and network aggregation, until Q no longer improves.
- 03 The algorithm is near-linear in time — suitable for million-node graphs.
- 04 Outputs a hierarchy of communities, not a flat partition.
- 05 Widely used in social networks, biology, finance, and NLP.

What is a K-Nearest Neighbor (KNN) Graph?



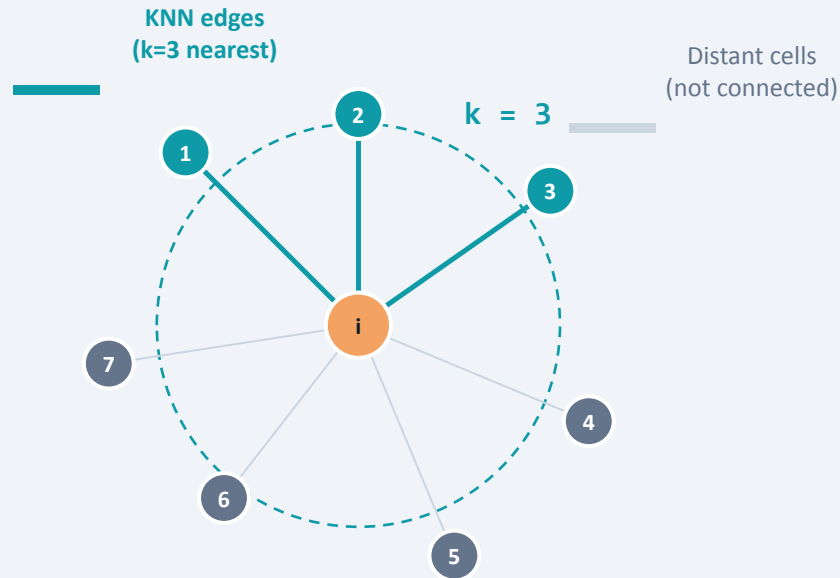
Definition

Given a set of **n cells**, each represented as a point in PCA space:

For every cell **i**, find its **k** most similar cells (by Euclidean distance).

Draw a directed edge from **i** to each of its **k** neighbors.

The result is the KNN graph: a sparse representation of local structure in high-dimensional expression space.



KNN Graph: Key Properties & Limitations



Distance metric

Seurat uses Euclidean distance in PCA space. The top N PCs capture most biologically meaningful variance, so distances there are more reliable than in raw gene expression space.



Directed vs. undirected

KNN is initially directed ($i \rightarrow j$ does not imply $j \rightarrow i$). Seurat symmetrizes it: an edge exists if either cell nominated the other as a neighbor.



Choosing k

Default is $k=20$. Larger $k \rightarrow$ smoother, more connected graph (favors larger clusters). Smaller $k \rightarrow$ sparser graph (can resolve rare cell types but more noise-sensitive).



Core limitation

Edge weights are binary (0 or 1). Two cells that share 1 neighbor out of 20 look the same as two cells sharing 19 of 20 neighbors. This discards valuable similarity information.



Uneven density problem

In heterogeneous tissues, dense populations (e.g. abundant T cells) and sparse ones (e.g. rare progenitors) have very different neighborhood structures. Binary KNN treats them equally.

From KNN to SNN: Refining Edge Weights



The core idea: two cells are more similar if they share many of the same neighbors — not just if they are each other's neighbors.

1

Build KNN graph

Connect each cell to its k nearest neighbors in PCA space.

2

Count shared neighbors

For every pair of connected cells (i, j) , count how many neighbors they have in common: $|N(i) \cap N(j)|$.

3

Compute Jaccard similarity

Divide shared neighbors by the union of their neighbor sets:
$$\text{Jaccard}(i, j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$$

4

Prune weak edges

Drop edges where Jaccard $<$ threshold (default 1/15 in Seurat). This removes spurious connections.

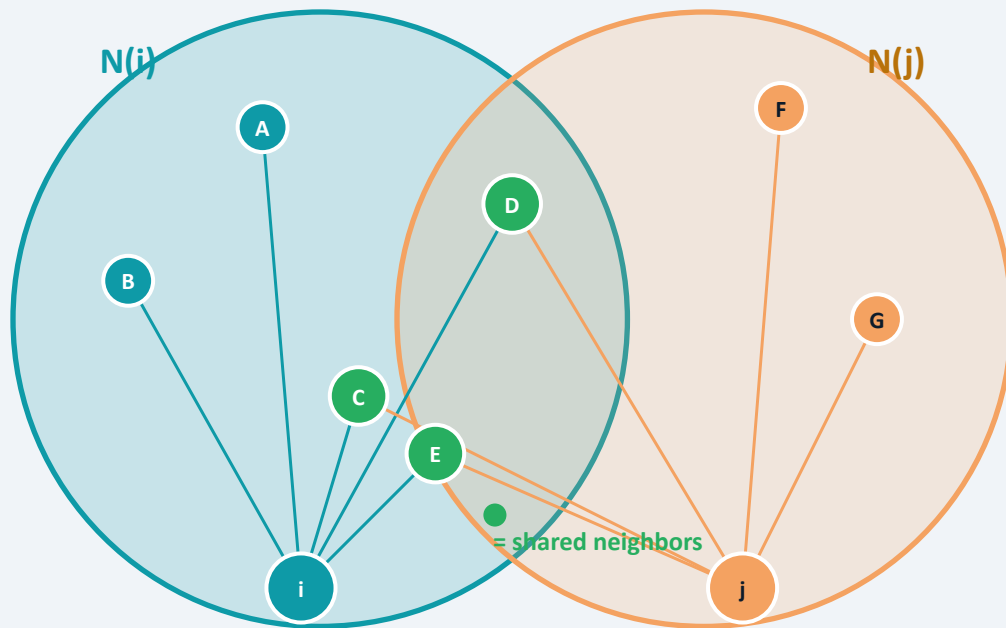
5

SNN graph ready

Remaining edges carry Jaccard weights (0–1). This is the input to Louvain/Leiden.

$$\text{Jaccard}(i, j) = \frac{|N(i) \cap N(j)|}{|N(i) \cup N(j)|}$$

Jaccard Similarity: A Worked Example



Calculation

$N(i) = \{A, B, C, D, E\}$

$N(j) = \{C, D, E, F, G\}$

Intersection:

$N(i) \cap N(j) = \{C, D, E\}$

→ 3 shared neighbors

Union:

$N(i) \cup N(j) = \{A, B, C, D, E, F, G\}$

→ 7 total neighbors

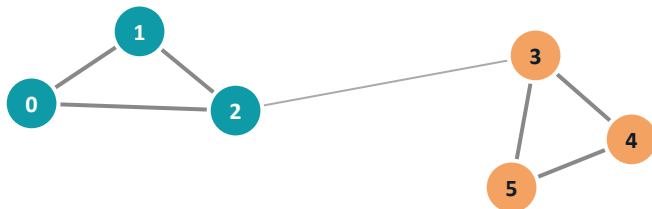
$Jaccard(i, j) = 3/7 \approx 0.43$

KNN vs. SNN: Why the Upgrade Matters



KNN Graph

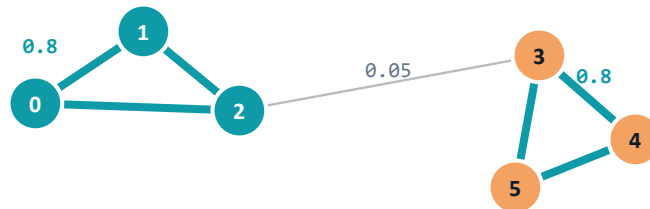
Binary edges — topology only



Bridge edge (2→3) looks identical
to strong intra-cluster edges

SNN Graph (Jaccard-weighted)

Weighted edges — similarity encoded



Bridge edge pruned (Jaccard=0.05 < threshold)
Strong intra-cluster edges clearly visible

Why SNN Works Well for scRNA-seq



01 Curse of dimensionality

scRNA-seq cells live in thousands of gene dimensions. In high dimensions, raw distances become unreliable — most cells end up roughly equidistant. Shared-neighbor counts are more robust because they rely on the rank ordering of distances, not their absolute values.

02 Dropout noise

A gene being zero in one cell but not another inflates Euclidean distance. If both cells genuinely belong to the same type, they'll still share most of the same neighbors. SNN smooths over this technical noise.

03 Uneven cluster densities

Real tissues have abundant cell types (T cells) and rare ones (progenitors). SNN naturally handles this: within a dense cluster, Jaccard scores are high; at boundaries, they drop sharply. This makes community boundaries cleaner.

04 Pruning removes spurious edges

The `prune.SNN` threshold (default 1/15) removes edges with near-zero Jaccard. This makes the graph sparser and faster to cluster, and reduces the chance of merging biologically distinct populations.

The Full Seurat Graph Pipeline



1 PCA Reduce 20,000+ genes to top N principal components (typically 10–50 PCs).

2 KNN For each cell, find k nearest neighbors in PCA space (Euclidean distance). Default k=20.

3 SNN Reweight KNN edges by Jaccard similarity of neighborhood overlap. Prune edges below threshold.

4 Louvain / Leiden Run community detection on weighted SNN graph. Resolution parameter controls granularity.

5 Clusters Each community = candidate cell type. Visualize with UMAP/tSNE; validate with marker genes.

FindNeighbors() builds steps 2–3. FindClusters() runs step 4.

Google AI experiment prompted by a student's question

I have 2 single cell experiments. one is immune cells from a healthy liver, and one is immune cells from NAFLD liver. How can I compare the cell populations and the genes expressed in these 2 samples

<https://share.google/aimode/ZTSeznWq3QQvRebfR>

give me an R script / qmd document that goes through all the steps above

<https://share.google/aimode/CcTAvQ3l8wVfbuUTI>